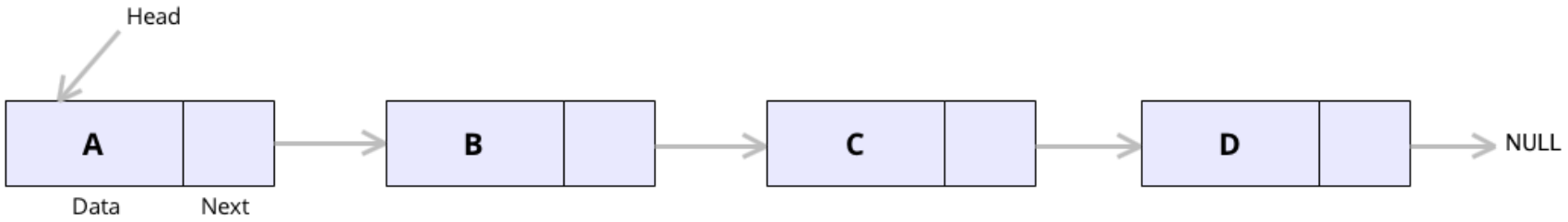
Linked List

DSA with C++

Introduction to Linked List

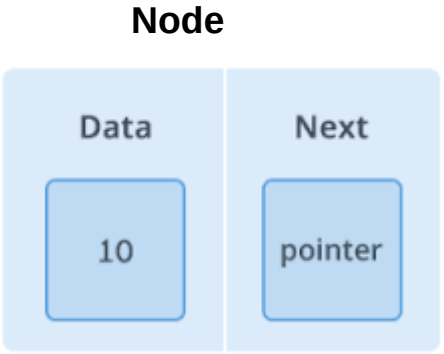
What is a Linked List(LL) in Data Structure?

- A linked list is a **linear dynamic data structure**, in which the elements are not stored at contiguous memory locations.
- A linked list is also a collection of nodes that contain a data part and a next pointer that contains the memory address of the next element in the list.
- A Linked List is a sequence of links which contains items. Each link contains a connection to another link.
- A linked list consists of nodes where each node contains a data field and a reference(link) to the next node in the list.



Terminologies of Linked List

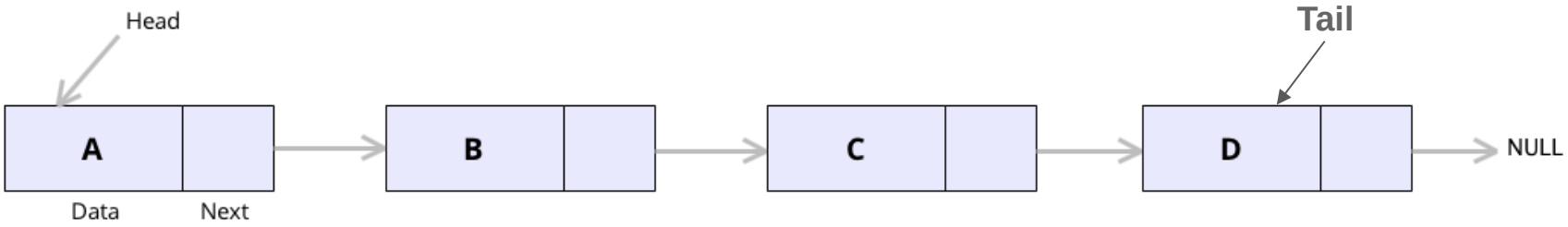
Linked list is the second most-used data structure after an array. Following are the important terms to understand the concept of Linked List.



- A **node** is **a collection of two sub-elements or parts**. A **data** part that stores the element and a **next** part that stores the link to the next node. The next part is a pointer that stores the address of the second link.
- **The Head:** The first node of a linked list ,
- **Tail**: The last node of the linked list.It is because no memory address is specified in the last node, the final node of the linked list will have a null next pointer.

Representation of A linked List

A linked list is represented by a pointer to the first node of the linked list. The first node is called the **head** of the linked list. If the linked list is empty, then the value of the head points to NULL. WIth the below illustration, following are the important points to be considered:



- Linked List contains a link element called first/head.
- Each link carries a data field(s) and a link field called next.
- Each link is linked with its next link using its next link.
- Last link carries a link as null to mark the end of the list.

In C++, we can represent a node using structures or classes. In C, structures can be used to represent Linked List.
In Java LinkedList can be represented as a class and a Node as a separate class. The LinkedList class contains a reference of Node class type.

Why Linked Lists?

Arrays can be used to store linear data of similar types, but arrays have the following limitations:

- **The size of the arrays is fixed**: So we must know the upper limit on the number of elements in advance. Also, generally, the allocated memory is equal to the upper limit irrespective of the usage.
- **Insertion of a new element / Deletion of a existing element in an array of elements is expensive:** The room has to be created for the new elements and to create room existing elements have to be shifted but in Linked list if we have the head node then we can traverse to any node through it and insert new node at the required position.

Example:

To maintain a sorted list of IDs in an array id[] = [10, 11, 12, 17, 20];

*If we want to insert a new ID **15**, then to maintain the sorted order, we have to move all the elements after 12.*

Deletion is also expensive with arrays until unless some special techniques are used. For example, to delete 11 in id[], everything after 11 has to be moved due to this so much work is being done which affects the efficiency of the code.

Advantages of Linked Lists over arrays

- Dynamic Array.
- Ease of Insertion/Deletion compared to contiguous (array) list.
- Overflow can never occur unless the memory is actually full

Disadvantages/Drawback of Linked Lists

- Pointers require extra space.
- Linked lists does not all random access. We have to access elements sequentially starting from the first node(head node). So we cannot do a binary search with linked lists efficiently with its default implementation.
- Time must be spend traversing and changing the pointers.
- Programming is typically trickie with pointers
- Reverse traversing is not possible in singly linked lists.
- Direct access to an element is not possible in a linked list as in an array by index.
- Searching for an element is costly and requires O(n) time complexity.
- Sorting of linked lists is very complex and costly.

Types of Linked Lists

Simple Linked List: We can move or traverse the linked list in only one direction. where the next pointer of each node points to other nodes but the **next pointer of the last node** points to **NULL**. It is also called **"Singly Linked List".**

Doubly Linked List We can move or traverse the linked list in both directions (Forward and Backward)

Circular Linked List : The last node of the linked list contains the link of the first/head node of the linked list in its next pointer.

Doubly Circular Linked List – A Doubly Circular linked list or a circular two-way linked list is a more complex type of linked list that contains a pointer to the next as well as the previous node in the sequence.

Header Linked List – A header linked list is a special type of linked list that contains a header node at the beginning of the list.

Basic Operations on Linked List

Following are the basic operations supported by a list:

- **Insertion** – Adds an element (at the beginning, end, any specified position) of the list.
- **Deletion** – Deletes an element at the beginning of the list.
- **Display** – Displays the complete list.
- **Search** – Searches for an element using the given key.
- **Delete** – Deletes an element using the given key.

NB: All this operations are performed while traversing a LL

Singly Linked List

A linked list is represented by a pointer to the first node of the linked list. The first node is called the head of the linked list. If the linked list is empty, then the value of the head points to NULL.

Each node in a list consists of at least two parts:

- A Data Item (we can store integers, strings, or any type of data).
- Pointer (Or Reference) to the next node (connects one node to another) or An address of anothe


```
// A linked list node
struct Node {
    int data;
    Node* next;
};
```

Or

```
class Node {
public:
    int data;
    Node* next;
}
```

Algorithm to display a singly Linked List

- **STEP 1:** SET PTR = HEAD
- **STEP 2:** IF PTR = NULL
- WRITE "EMPTY LIST"

  GOTO STEP 7

  END OF IF**STEP 4:** REPEAT STEP 5 AND 6 UNTIL PTR = NULL
- **STEP 5:** PRINT PTR→ DATA
- **STEP 6:** PTR = PTR → NEXT
- [END OF LOOP]**STEP 7:** EXIT

```
void printList(Node* head)
{
    while (head != NULL) {
        cout << head->data <<
" ";
        head = head->next;
    }
cout<<”NULL”;
}
```

C++ Program using class

```
// A simple C++ program for
// traversal of a linked list
#include <bits/stdc++.h>
using namespace std;
class Node {
public:
    int data;
    Node* next;
};
// This function prints contents of
//  linked list
starting from the given node
void printList(Node* n)
{
    while (n != NULL) {
        cout << n->data << " ";
        n = n->next;
    }
}
```

```
 // Driver's code
int main()
{
    Node* head = NULL;
    Node* second = NULL;
    Node* third = NULL;

    // allocate 3 nodes in the heap
    head = new Node();
    second = new Node();
    third = new Node();
    head->data = 1; // assign data in first node
    head->next = second; // Link first node with
second
    second->data = 2; // assign data to second node
    second->next = third;
    third->data = 3; // assign data to third node
    third->next = NULL;
    // Function call
    printList(head);
    return 0;
}
```

C++ Program with Structures

```
// A simple C++ program for
// traversal of a linked list
#include <bits/stdc++.h>
using namespace std;
struct Node {
public:
    int data;
    Node* next;
};
// This function prints contents of
//  linked list
starting from the given node
void display(Node* n)
{
    while (n != NULL) {
        cout << n->data << " ";
        n = n->next;
    }
}
```

```
 // Driver code
int main()
{
   Node* root = NULL;
    Node* second = NULL;
    Node* third = NULL;

    // allocate 3 nodes in the heap
    root = new Node();
    second = new Node();
    third = new Node();

    root->data = 1; // assign data in first node
    root->next = second; // Link first node with second
    second->data = 2; // assign data to second node
    second->next = third;
    third->data = 3; // assign data to third node
    third->next = NULL;

    display(root);
    return 0;
}
```

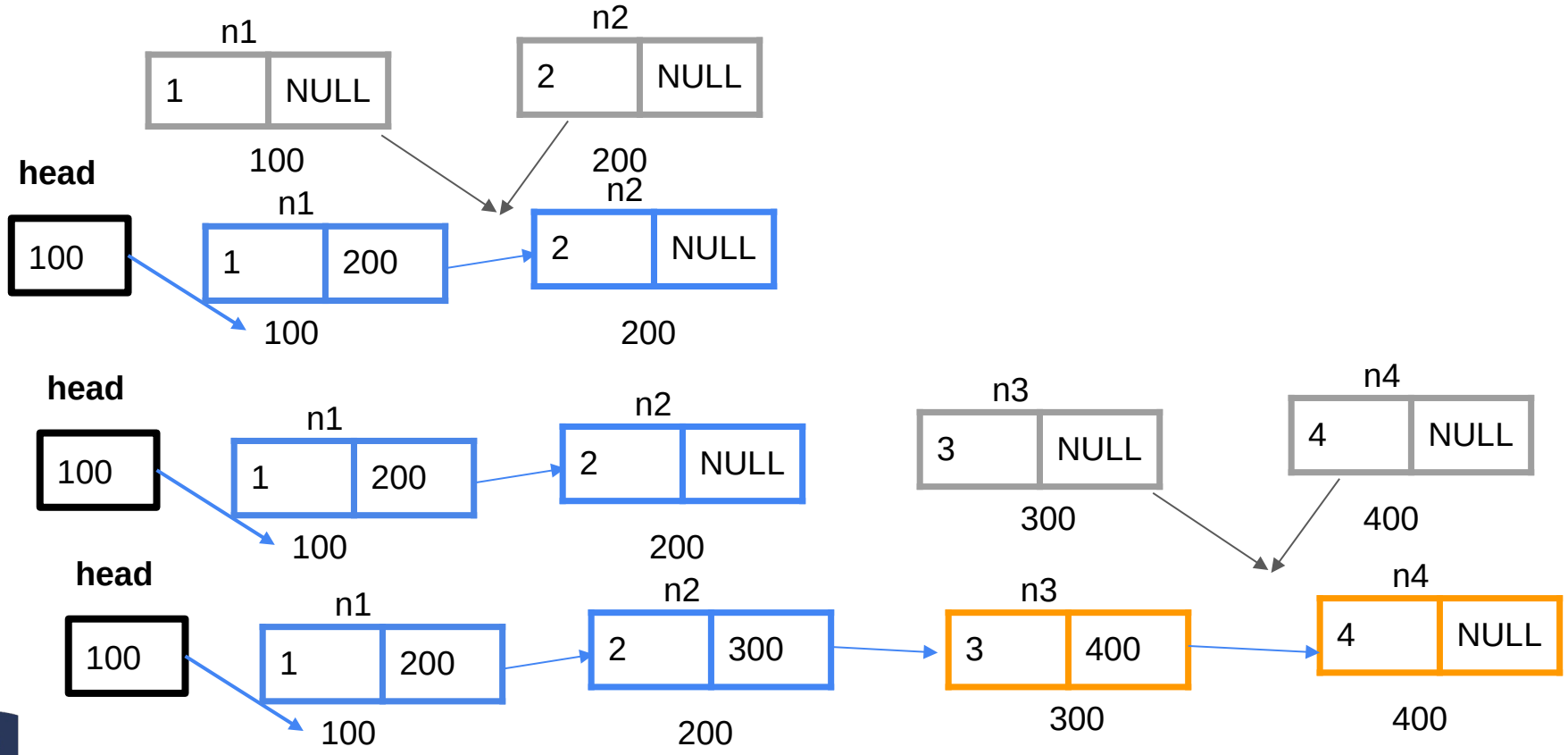
Note: In C, You need to allocate memory to individual node using malloc ()

Linked list using Class with a constructor

```
#include<iostream>
using namespace std;

class Node {
   public:
     int data;
     Node *next;
     Node(int data){
       this->data = data;
       next = NULL;
     }
};
void display(Node* n)
{
    while (n != NULL) {
        cout << n->data << " ";
        n = n->next;
    }
}
```

```
int main(){
   /// Statically
    Node n1(1);
    Node n2(2);
//To create a linked list n1.next  should be  the address of n2
  n1.next = &n2;
  cout<<n1.data<<" "<<n2.data<<endl;
//Head is not a node but a pointer to the first node
  Node *head = &n1;
  cout<<head->data<<endl;
 //Dynamically
Node *n3=new Node(3);
Node *n4=new Node(4);
//Connect the second node and the third node
  n2.next=n3;
//Link the third node and the fourth node
n3->next=n4;
display(head);
   return 0;
}
```

Illustration of Linked list using Class with a constructor



Multiple data members in a linked list

```
#include<iostream>
using namespace std;

class Student {
   public:
     int id;
     int age;
     string name;
     Student *next;
     Student(int id,int age, string name){
       this->id = id;
       this->age=age;
       this->name=name;
       next = NULL;
     }
};
void display(Student* n)
{
    while (n != NULL) {
        cout << n->id << " "<<n->name<<" "<<n-
>age<<endl;
        n = n->next;
    }
}
```

```
int main(){
    /// Statically
    Student n1(1,20,"John");
    Student n2(2,16,"Michaella");
//To create a linked list n1.next  should be  the address of n2
  n1.next = &n2;
//Head is not a node but a pointer to the first node
  Student *head = &n1;
  //Dynamically
Student *n3=new Student(3,17,"Peter");
Student *n4=new Student(4,18,"Maureen");
//COnnect the second node and the third node
  n2.next=n3;
  //Link the third node and the fourth node
n3->next=n4;
display(head);
    return 0;
}
```

```
1 John 20
2 Michaella 16
3 Peter 17
4 Maureen 18
```

Counting elements of a list

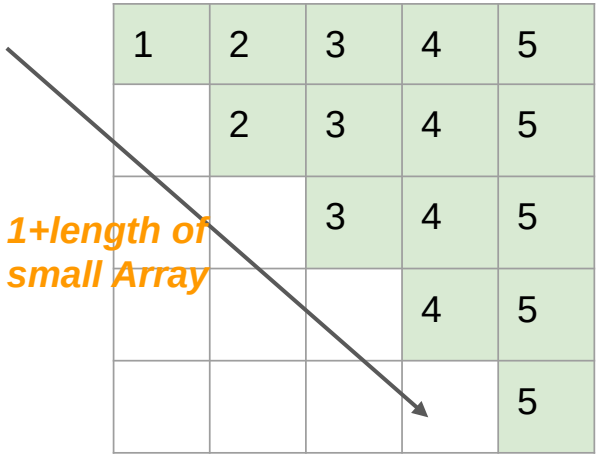
We can use the variable count to count the number of items in our linked list.

If the head is not equal to null, the count variable will keep incrementing and moving the head to the next element.

```
int length(Node *head){
    int count = 0;
    while(head){
        count++;
        head = head->next;
    }
    return count;
}
```

Counting/Length of LL Using recursion

```
int length(Node *head){
    if(head==NULL)
return 0;
//Return 1 + the length of the small LL
   return 1+length(head->next);
}
```



INSERT A NEW NODE IN A LINKED LIST

Insertion of a new Node at the Beginning

Assume that the linked list has elements: 20 30 40 NULL

If we insert 100, it will be added at the beginning of a linked list.

After insertion, the new linked list will be

100 20 30 40 NULL

Algorithm

1. Declare a head pointer and make it as NULL.
2. Create a new node with the given data.
3. Make the new node points to the head node.
4. Finally, make the new node as the head node.

Declare a head pointer and make it as NULL

```
struct Node

{
    int data;
    Node *next;
};
    Node *head = NULL;
```

Create a new node with the given data.

```
void addFirst(Node **head, int val)
{
      Node *newNode = new Node();
      newNode->data = val;
    };
```

Make the newnode points to the head node

```
void addFirst(Node **head, int val)
{
      //create a new node
      Node *newNode = new Node();
      newNode->data = val;
      newNode->next = *head;
   }
```

Make the new node as the head node.

```
void addFirst(Node **head, int val)
{
      //create a new node
      Node *newNode = new Node();
      newNode->data = val;
      newNode->next = *head;
      *head = newNode;
}
```

Adding a node at the beginning (method 1)

```
// A  C++ program for
// insert at head of a linked list
#include <bits/stdc++.h>
using namespace std;
struct Node {
public:
    int data;
    Node* next;
Node(int data){
this->data=data;
this->next=NULL;
}
};
Node *head = NULL;

//Here changes to head are reflected outside the function.
void addFirst(Node **head, int val)
{
      //create a new node
      Node *newNode = new Node(val);
      newNode->next = *head;
      *head = newNode;
```

```
void display(Node* n)
{
    while (n != NULL) {
        cout << n->data << " ";
        n = n->next;
    }
}
 // Main
int main()
{
   Node* root = NULL;
    Node* second = NULL;
    Node* third = NULL;
    root = new Node(1);
    second = new Node(2);
    third = new Node(3);
    root->next=second;
    second->next=third;
    head=root;
    addFirst(&head,20);
    addFirst(&head,30);
    display(head);
    return 0;
}
```

Adding a node at the beginning method 2(alternative)

```
// A  C++ program for
// insert at head of a linked list
#include <bits/stdc++.h>
using namespace std;
struct Node {
public:
    int data;
    Node* next;
Node(int data){
this->data=data;
this->next=NULL;
}
};

Node* insertAtHead(Node *head, int data){

        Node* n = new Node(data);
        n->next = head;
        head = n;
        return head;
}
```

```
void display(Node* n)
{
    while (n != NULL) {
        cout << n->data << " ";
        n = n->next;
    }
}
 // Main
int main()
{
   Node* root = NULL;
    Node* second = NULL;
    Node* third = NULL;
    root = new Node(1);
    second = new Node(2);
    third = new Node(3);
    root->next=second;
    second->next=third;
    head=root;
    head=insertAtHead(head,25);
    display(head);
    return 0;
}
```

Adding a node at the beginning method 2(alternative)

```
// A  C++ program for
// insert at head of a linked list
#include <bits/stdc++.h>
using namespace std;
struct Node {
public:
    int data;
    Node* next;
Node(int data){
this->data=data;
this->next=NULL;
}
};

Node* insertAtTail(Node *head, int data){
        Node *tail = head;
  if(head == NULL){
       head = n;
       tail = n;
    }else {
       tail -> next = n;   /// Inserting at Tail
       tail = n;
    }
       return head;
}
```

```
void display(Node* n)
{
    while (n != NULL) {
        cout << n->data << " ";
        n = n->next;
    }
}
 // Main
int main()
{
   Node* root = NULL;
    Node* second = NULL;
    Node* third = NULL;
    root = new Node(1);
    second = new Node(2);
    third = new Node(3);
    root->next=second;
    second->next=third;
    head=root;
    head=insertAtHead(head,25);
    display(head);
    return 0;
}
```

Taking input of a Linked List and Insert at Head

Take user data and make a linked list by adding new elements at the head(first node) dynamically.

The User will stop if the user add -1 and the the Linked list is printed,

```
#include<iostream>
using namespace std;

class Node {
   public:
     int data;
     Node *next;

     Node(int data){
       this->data = data;
       next = NULL;
     }
};

void print(Node *head){
    Node *temp = head;
    while(temp!=NULL){
        cout<<temp->data<<"-
>";
        temp = temp->next;
    }
    cout<<"NULL"<<endl;
}
```

```
Node* UserData(){
   int data;
   cin>>data;
   Node *head = NULL;  /// LL is empty
   Node *tail = NULL;   /// LL is empty
   while(data != -1){
    /// creating LL
    Node *n = new Node(data);
    /// 1st node
    if(head == NULL){
        head = n;
        tail = n;
    }else {
        n -> next = head;   /// Inserting at head
        head = n;
    }
    cin>>data;
   }
   return head;
}
int main(){
   Node *head = userData();
   print(head);
   return 0;
}
```

Taking input of a Linked List and Insert at Tail

Take user data and make a linked list by adding new elements at the tail(last node) dynamically.

The User will stop if the user add -1 and the the Linked list is printed,

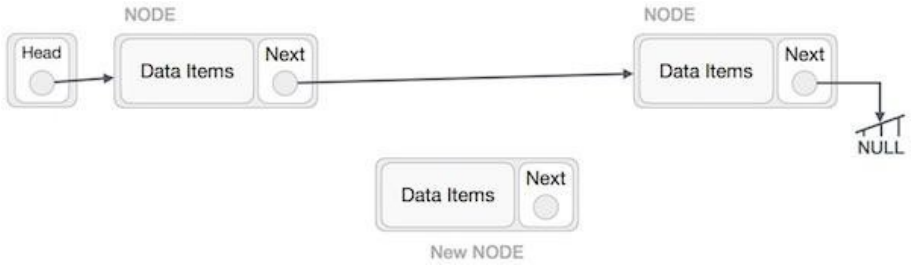
```
#include<iostream>
using namespace std;

class Node {
   public:
     int data;
     Node *next;

     Node(int data){
       this->data = data;
       next = NULL;
     }
};
void print(Node *head){
    Node *temp = head;
    while(temp!=NULL){
  cout<<temp->data<<"->";
        temp = temp->next;
    }
    cout<<"NULL"<<endl;
}
```

```
Node* userData(){
   int data;
   cin>>data;
   Node *head = NULL;  /// LL is empty
   Node *tail = NULL;   /// LL is empty
   while(data != -1){
    /// creating LL
    Node *n = new Node(data);
    /// 1st node
    if(head == NULL){
        head = n;
        tail = n;
    }else {
        tail -> next = n;   /// Inserting at Tail
        tail = n;
    }
    cin>>data;
  }
   return head;

}
int main(){
   Node *head = userData();
   print(head);
   return 0;
}
```

Insertion Operation at any position

a) Adding a new node in the linked list is a more than one step activity. We shall learn this with diagrams here. First, create a node using the same structure and find the location where it has to be inserted.Image(1)

image (1)



b) Imagine that we are inserting a node B (NewNode), between A (LeftNode) and C (RightNode). Then point B.next to C –

NewNode.next =RightNode;

Image (2)

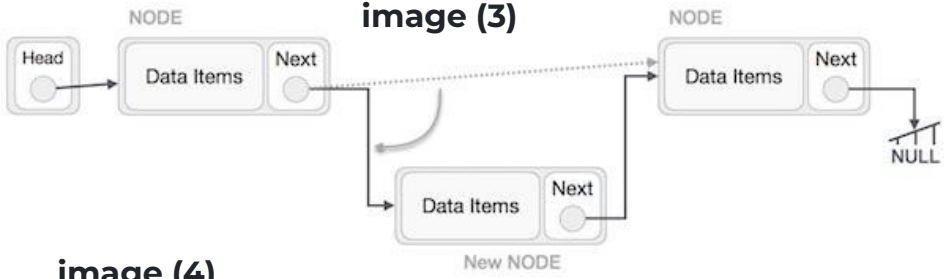
image (2)

c) Now, the next node at the left should point to the new node.

LeftNode.next = NewNode;

Image(3)

image (3)



d) This will put the new node in the middle of the two.

image (4)

Exercises

1. Perform an insert operation on singly linked list at any position.
    a. Using Structures
    b. Using classes

Insert at any Position in a Linked List

```
// A  C++ program for
// insert at any position of a linked list
#include <bits/stdc++.h>
using namespace std;
struct Node {
public:
    int data;
    Node* next;
Node(int data){
this->data=data;
this->next=NULL;
}
};


Node* insertAtHead(Node *head, int data){

        Node* n = new Node(data);
        n->data=data;
        n->next = head;
        head = n;
        return head;
}
```

```
Node* insertAtPos(Node *head, int i, int data)
{
      if(i<0){
        return head;
      }
      if(i==0){
        Node* n = new Node(data);
        n->next = head;
        head = n;
        return head;
      }
      Node *temp = head;
      int count = 1;
      while(count<=i-1 && head!=NULL){
        head = head->next;
        count++;
      }
      if(head){
        Node *n = new Node(data);
        n->next = head->next;
        head->next = n;

        return temp;
      }
      return temp;
}
```

```
void display(Node* n)
{
    while (n != NULL) {
        cout << n->data << " ";
        n = n->next;
    }
cout<<endl;
}
 // Main
int main()
{
   Node* root = NULL;
    Node* second = NULL;
    Node* third = NULL;
    root = new Node(1);
    second = new Node(2);
    third = new Node(3);
    root->next=second;
    second->next=third;
   Node *head=root;
    cout<<"Initial "<<endl;
    display(head);
    head=insertAtPos(head,2,250);
    display(head);
    cout<<"After adding at 2"<<endl;
     head=insertAtPos(head,4,200);
    display(head);
    cout<<"After adding at 4"<<endl;
     head=insertAtPos(head,0,150);
    display(head);
    cout<<"After adding at 0"<<endl;
    head=insertAtPos(head,6,550);
    display(head);
    cout<<"After adding at 6"<<endl;
    return 0;
}
```

Output

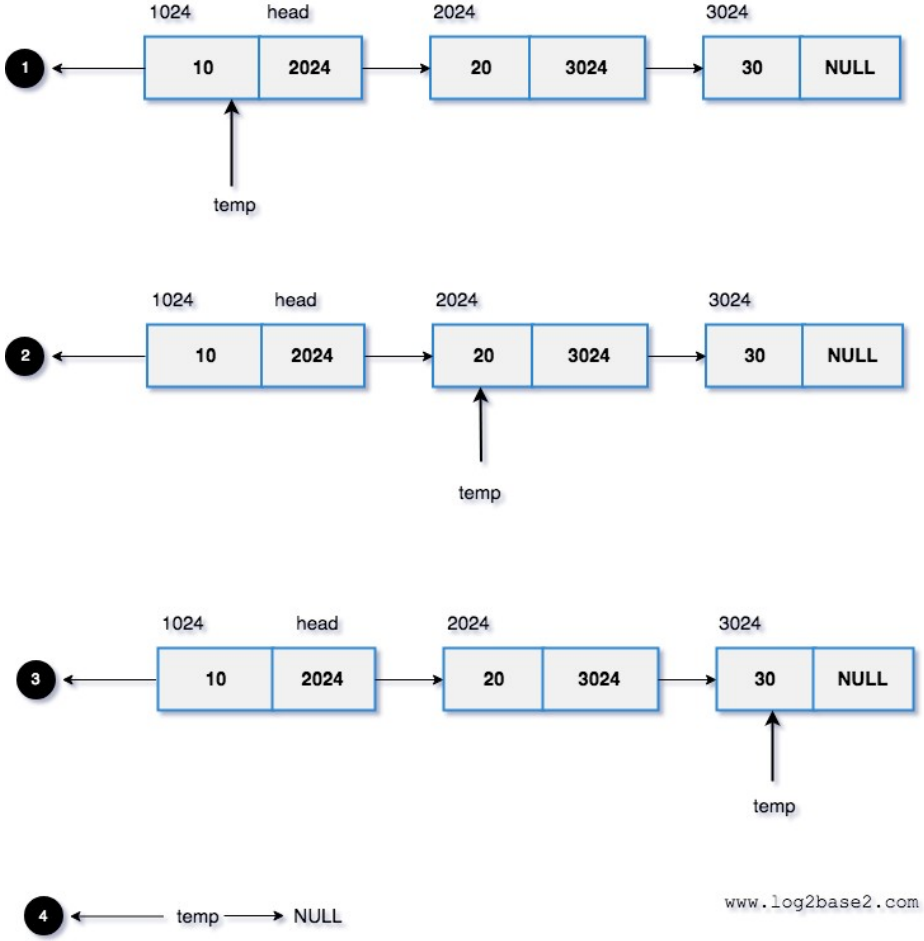
Initial
1 2 3
1 2 250 3
After adding at 2
1 2 250 3 200
After adding at 4
150 1 2 250 3 200
After adding at 0
150 1 2 250 3 200 550
After adding at 6

Searching in a linked List

```
1) Initialize a node pointer, current =
head.
2) Do following
while current is not NULL
a) current -> data is equal to the key
being searched
return true.
b) current = current -> next
3) Return false
```

C++ Implementation

```
bool search(Node * head, int
x) {
  Node * current = head; //
Initialize current
  while (current != NULL) {
    if (current -> data == x)
      return true;
    current = current -> next;
  }
  return false;
}
```



Searching: Get a position of the node with the key

You can also return the position of the node you are searching.

If the key is not found in the linked List, the value zero (0) will be returned

```
int search2(Node *head, int key){
int index=1;
while(head!=NULL){
if(head->data==key){
return  index;
}
head=head->next;
index++;
}
// If the key is not found 0 will be returned.
return 0;
}
```

main

```
int position=search2(head,6);
(position>0)? cout<<"The key  is found at "<<position<<" node": cout<<"The key is not
Found";
if(position>0)
cout<<"The key  is found at "<<position<< " node";
else
cout<<"The key is not Found";
```

Delete in the linked List

Delete from a Linked List:-

You can delete an element in a list from:

- Beginning
- End
- Middle

Delete at the beginning

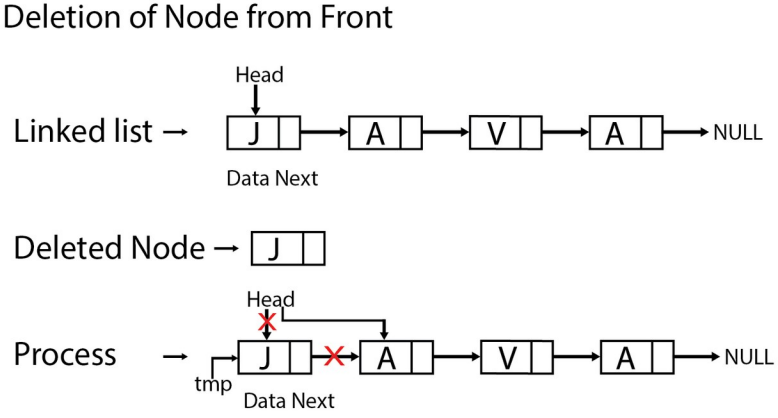
Deleting a node from the beginning of the list is the simplest operation of all. It just needs a few adjustments in the node pointers. Since the first node of the list is to be deleted, therefore, we just need to make the head point to the next of the head. This will be done by using the following statements.

```
Keep the head in a temporary pointer
Point head to the next node i.e. second node
    temp = head
    head = head->next
```

```
Make sure to free unused memory
      free(temp); or delete temp;
```

main

```
head=deleteFirstNode(head);
display(head);
```




```
Node* deleteFirstNode(Node* head){
if (head == NULL)
        return NULL;
    // Move the head pointer to the next
node
    Node* temp = head;
    head = temp->next;
    delete temp;
    return head;
}
```

Delete at the end

Algorithm:

1. If the first node is null or there is only one node, then they
return NULL.

```
if headNode == NULL then
return NULL
if headNode->nextNode == NULL then free
head and
return NULL
```

2. Create an extra space secondLast, and traverse the linked list till the second last node.

```
while secondLast->next->next != null
secondLast = secondLast->next
```

3. delete the last node, i.e.the next node of the second last node delete(secondLast->nextNode), and set the value of the next second - last node to null

```
Node* deleteLastNode(Node* head)
{
    if (head == NULL)
        return NULL;

    if (head->next == NULL) {
        delete head;
        return NULL;
    }

    // Find the second last node
    Node* second_last = head;
    while (second_last->next->next !=
NULL)
        second_last = second_last->next;

    // Delete last node
    delete (second_last->next);
    // Change next of second last
    second_last->next = NULL;

    return head;
}
```

Delete an Element in the Linked List based on the head and position i

Reach the element before the element to delete

```
Node* deleteNode(Node* head, int i){
   if(i<0){
      return head;
   }
   if(i==0 && head){
//Store the address of the next node
    Node* newHead = head->next;
//Isolate the current node
    head->next = NULL;
//Delete the current head node
    delete head;
//Start the LL with the new Head
    return newHead;
   }
//Keep head in a temp curr variable
  Node* curr = head;
   int count = 1;
//Before the target, if the current pointer is valid change the
current to the next of current
   while(count<i-1 && curr!=NULL){
    curr = curr->next;
    count++;
   }
    if(curr && curr->next){
//Keep the address of element to delete
     Node *temp = curr->next;
//Change the next of current to the temp
current
     curr->next = curr->next->next;
//Deallocate the temp memory
     temp->next=NULL;
//Delete the temp
     delete temp;
//return the head
     return head;
   }
   return head;
}
```

Reversing a Linked List

Given a pointer to the head node of a linked list, the task is to reverse the linked list. We need to reverse the list by changing the links between nodes.

Examples:

***Input**: Head of following linked list*
1->2->3->4->NULL
***Output**: Linked list should be changed to,*
4->3->2->1->NULL
***Input**: Head of following linked list*
1->2->3->4->5->NULL
***Output**: Linked list should be changed to,*
5->4->3->2->1->NULL
input: NULL
Output: NULL
input: 1->NULL
Output: 1->NULL

Reversing a Linked List

*The idea is to use three pointers **curr**, **prev,** and **nextNode** to keep track of nodes to update reverse links.*

Follow the steps below to solve the problem:

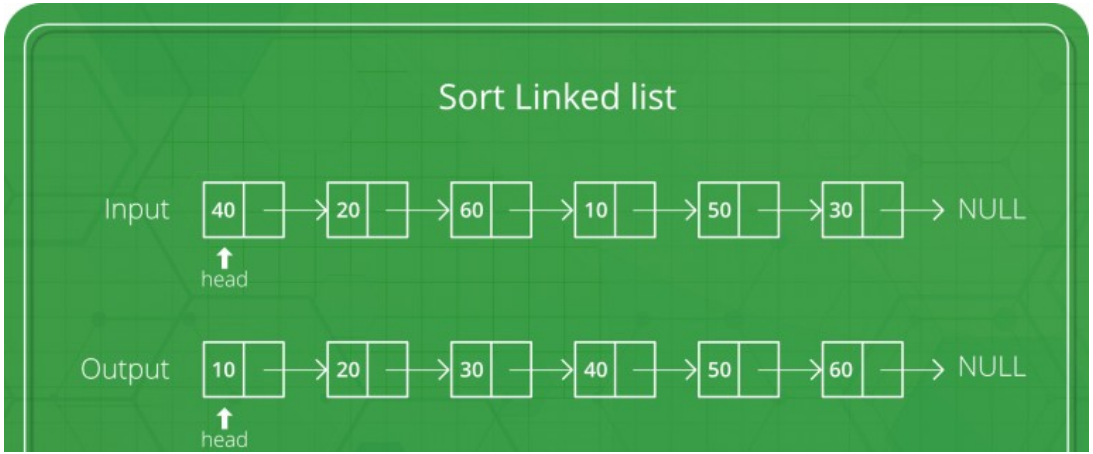
- Initialize three pointers **prev** as NULL, **curr** as **head**, and **nextNode** as NULL.
- Iterate through the linked list if current is NOT NULL. In a loop, do the following:
    - Before changing the **next** of **curr**, store the **nextNode** node
        - nextNode = curr -> next
    - Now update the **nextNde** pointer of **curr** to the **prev**
        - curr -> next = prev
    - Update **prev** as **curr** and **curr** as **next**Node
        - prev = curr
        - curr = nextNode
- Return prev

Reversing a Linked List:Implementation

```
Node*  reverse(Node* head){
//Make head our current node
        Node *current=head;
        //Make previous pointing to NULL
        Node *previous=NULL;
                Node *n=NULL;
        //We need to loop until current or head pointing to NULL
        while (current!=NULL){
        //Save the next element
                n=current->next;
           //Instead of pointing to next element, point to previous
                current->next=previous;
                // Move pointers one position ahead.
                previous=current;
                current=n;
        }
     //The last prev is our head
        return previous;
}
```

Sorting a Linked List

Merge sort is a divide and conquer algorithm. It implies that we will implement the solution by breaking down the problem into various subproblems of a similar kind. We keep on repeating the same process until we hit a predefined base case. A base is a problem that we can solve using constant time and space resources. Below are the steps to merge and sort a linked list. Merge sort is often preferred for sorting a linked list.



Algorithm

1. Divide: Divide the linked list into two parts about its mid-point.
    - *Node *mid = mid_point(head);*
    - Now, divide point to the head by: *Node *a = head;*
    - This pointer will point to the mid_point: *Node *b = mid->next;*
    - To evaluate the mid-point of the linked list, use the fast and slow pointers approach**(Floyd's Tortoise and Hare algorithm)**
2. Sort: Sort the smaller linked lists recursively
    - *merge_sort(a);*
    - *merge_sort(b);*
3. Merge: Merge the sorted linked lists.
    - *merge(start, mid);*
    - This function will take two linked lists as input. Then it will recursively merge

Merge Two Lists

```
Node *merge(Node *a, Node *b)
{
    // base case
    if(a == NULL)
        return b;
    if(b == NULL)
        return a;

    // recursive case
    // take a head pointer
    Node *c;

    if(a->data < b->data)
    {
        c = a;
        c->next = merge(a->next, b);
    }
    else
    {
        c = b;
        c->next = merge(a, b->next);
    }

    return c;
}
```

Merge Sort

```
Node *mid_point(Node *head)
{
    // base case
    if(head == NULL || head->next == NULL)
        return head;
    // recursive case
    Node *fast = head;
    Node *slow = head;
    while(fast != NULL && fast->next != NULL)
    {
        fast = fast->next;
        if(fast->next == NULL)
            break;
        fast = fast->next;
        slow = slow->next;
    }
    return slow;
}
```

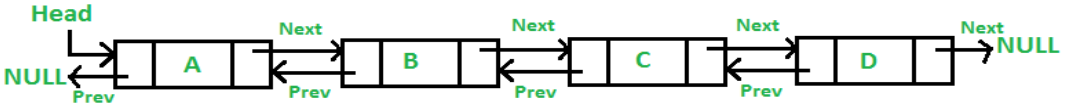
```
Node* merge_sort(Node *head)
{
    // base case
    if(head == NULL || head->next == NULL)
        return head;

    // recursive case
    // Step 1: divide the linked list into
    // two equal linked lists
    Node *mid = mid_point(head);
    Node *a = head;
    Node *b = mid->next;

    mid->next = NULL;

    // Step 2: recursively sort the smaller
    // linked lists
    a = merge_sort(a);
    b = merge_sort(b);

    // Step 3: merge the sorted linked lists
    Node *c = merge(a, b);

    return c;
}
```

Time Complexity Of Linked List

1. Insert: O(1), if inserted on head ;O(N), elsewhere
2. Delete: O(1), if deleted on head; O(N), elsewhere
3. Access: O(N)
4. Search:O(N)

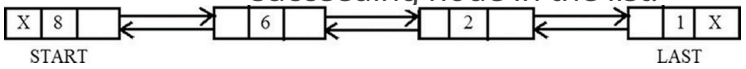
Doubly Linked List

A doubly linked list is a data structure which consists of nodes which have data, a pointer to the next node, and also a pointer to the previous node. his allows for efficient traversal of the list in **both directions**, making it suitable for applications where frequent **insertions** and **deletions** are required.



In a doubly linked list each node is divided into three parts:

1. The first part is called the previous pointer, which contains the address of the previous node in the list.
2. The second part is called the information part, which contains the information of the node.
3. The third part is called the next pointer, which contains the address of the succeeding node in the list.



Doubly Linked list

```
struct Node {

    int data;
    // Link/Pointer to  the Previous Element
    Node* prev;
    // Link/pointer to the Next Element
    Node* next;
    // Constructor
    Node(int d) {
       data = d;
       prev = next = NULL;
    }
};
```

Doubly Linked List

1. The first node of the linked list will contain a NULL value in the previous pointer to indicate that there is no element preceding in the list.
2. Similarly, the last node will also contain a NULL value in the next pointer field to indicate that there is no element succeeding it in the list.
3. Doubly linked lists can be traversed in both directions.

Various operations can be performed on a doubly linked list, which include:

4. Inserting a New Node in a Doubly Linked List
5. Deleting a Node from a Doubly Linked List
6. Traversal in Doubly Linked List
    - Forward
    - Backward
7. Searching in Doubly Linked List
8. Finding Length of Doubly Linkedlist

Forward Traversal

- Initialize a pointer to the head of the linked list.
- While the pointer is not null:
  - Visit the data at the current node.
  - Move the pointer to the next node.

```
void forwardTraversal(Node* head)
{
    Node* curr = head;
    while (curr != NULL) {
        cout << curr->data << " ";
        curr = curr->next;
    }
    cout << endl;
}
```

Backward Traversal

- Initialize a pointer to the tail of the linked list.
- While the pointer is not null:
- Visit the data at the current node.
- Move the pointer to the previous node.

```
void backwardTraversal(Node* tail)
{
    Node* curr = tail;
   while (curr != NULL) {
            cout << curr->data << " ";
           curr = curr->prev;
   }
   cout << endl;
}
```

Length of Doubly LL

To find the length of doubly list, we can use the following steps:

- Start at the head of the list.
- Traverse through the list, counting each node visited.
- Return the total count of nodes as the length of the list.

```
int findLength(Node * head) {
    int count = 0;
    for (Node * cur = head; cur != NULL;cur = cur -> next)
        count++;
    return count;
}
```

Insert at the beginning

To insert a new node at the beginning of the doubly list, we can use the following steps:

- Create a new node, say **new_node** with the given data and set its previous pointer to null, **new_node->prev = NULL**.
- Set the next pointer of new_node to current head, **new_node->next = head.**
- If the linked list is not empty, update the previous pointer of the current head to new_node, **head->prev = new_node**.
- Return new_node as the head of the updated linked list.

```
Node* insertBegin(Node* head, int data) {
    Node* new_node = new Node(data);
    new_node->next = head;
    if (head != NULL) {
        head->prev = new_node;
        head=new_node;
    }
    return head;
}
```

Insert at the End

To insert a new node at the end of the doubly linked list, we can use the following steps:

- Allocate memory for a new node and assign the provided value to its data field.
- Initialize the next pointer of the new node to nullptr.
- If the list is empty:
  - Set the previous pointer of the new node to nullptr.
  - Update the head pointer to point to the new node.
- If the list is not empty:
  - Traverse the list starting from the head to reach the last node.
  - Set the next pointer of the last node to point to the new node.
  - Set the previous pointer of the new node to point to the last node.

```
Node *insertEnd(Node *head, int new_data) {
    Node *new_node = new Node(new_data);
    if (head == NULL) {
        head = new_node;
    }
    else {
          Node *curr = head;
        while (curr->next != NULL) {
            curr = curr->next;
        }
        curr->next = new_node;
        new_node->prev = curr;
    }
    return head;
}
```

Delete at the Beginning

To delete a node at the beginning in doubly linked list, we can use the following steps:

- Check if the list is empty, there is nothing to delete. Return.
- Store the head pointer in a variable, say **temp**.
- Update the head of linked list to the node next to the current head, **head = head->next**.
- If the new head is not NULL, update the previous pointer of new head to NULL, **head->prev = NULL**.

```
Node *delHead(Node *head) {
    if (head == nullptr)
        return nullptr;
    Node *temp = head;
    head = head->next;
    if (head != nullptr)
        head->prev = nullptr;
    delete temp;
    return head;
}
```

Delete at the end of DLL

To delete a node at the end in doubly linked list, we can use the following steps:

- Check if the doubly linked list is empty. If it is empty, then there is nothing to delete.
- If the list is not empty, then move to the last node of the doubly linked list, say **curr**.
- Update the second-to-last node's next pointer to NULL, **curr->prev->next = NULL**.
- Free the memory allocated for the node that was deleted.

```
Node *delLast(Node *head) {
    if (head == NULL)
        return NULL;
    if (head->next == NULL) {
        delete head;
        return NULL;
    }
    // Traverse to the last node
    Node *curr = head;
    while (curr->next != NULL)
        curr = curr->next;

    // Update the previous node's next pointer
    curr->prev->next = NULL;
                // Delete the last node
    delete curr;
    return head;
}
```

Doubly Linked List(Task)

1. The first node of the linked list will contain a NULL value in the previous pointer to indicate that there is no element preceding in the list.
2. Similarly, the last node will also contain a NULL value in the next pointer field to indicate that there is no element succeeding it in the list.
3. Doubly linked lists can be traversed in both directions.

Various operations can be performed on a doubly linked list, which include:

4. Inserting a New Node in a Doubly Linked List
5. Deleting a Node from a Doubly Linked List

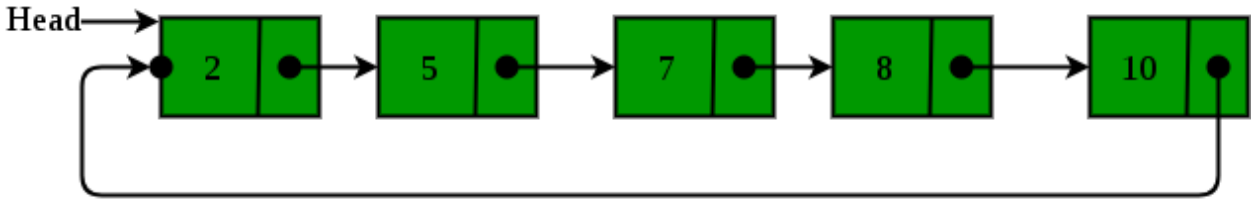
When a new node is inserted into an already existing doubly linked list.
We consider four cases for the insertion process in a doubly linked list.

6. A new node is inserted at the beginning.
7. A new node is inserted at the end.
8. A new node is inserted after a given node.
9. A new node is inserted before a given node.

Task: Implement all the above operations on a doubly linked list

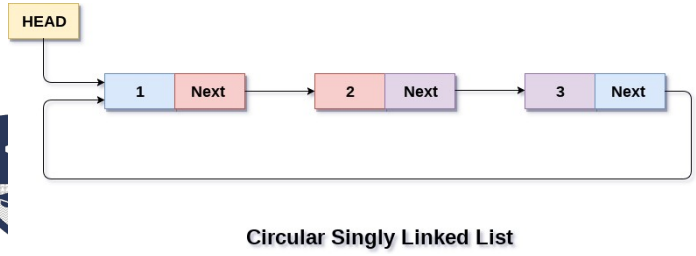
Circular Linked List

Circular linked list is a linked list where all nodes are connected to form a circle.There is no NULL at the end.A circular linked list can be a singly circular linked list or doubly circular linked list.



In a circular Singly linked list, the last node of the list contains a pointer to the first node of the list.
We can have circular singly linked lists as well as circular doubly linked lists.

We traverse a circular singly linked list until we reach the same node where we started. The circular singly linked list has no beginning and no ending. There is no null value present in the next part of any of the nodes.



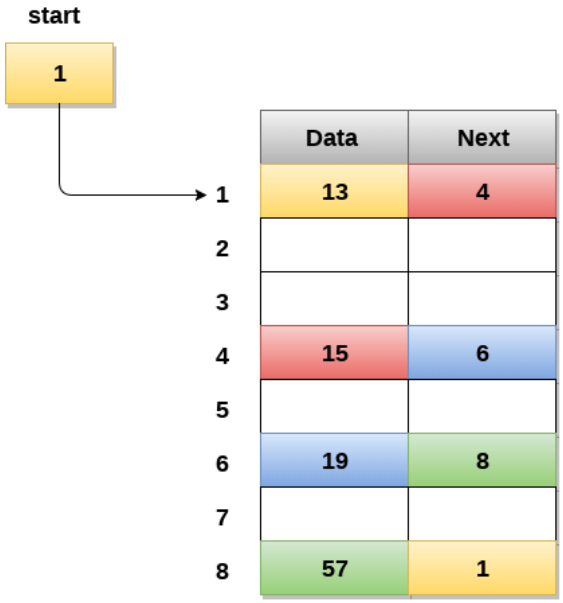
Circular Singly Linked List

Circular linked lists are mostly used in task maintenance in operating systems. There are many examples where circular linked lists are being used in computer science including browser surfing where a record of pages visited in the past by the user, is maintained in the form of circular linked lists and can be accessed again on clicking the previous button.

Memory representation of a circular Linked List

In the following image, memory representation of a circular linked list containing marks of a student in 4 subjects. However, the image shows a glimpse of how the circular list is being stored in the memory. The start or head of the list is pointing to the element with the index 1 and containing 13 marks in the data part and 4 in the next part. Which means that it is linked with the node that is being stored at the 4th index of the list.

However, due to the fact that we are considering a circular linked list in the memory therefore the last node of the list contains the address of the first node of the list.



Memory Representation of a circular linked list

Operations on Circular Singly linked list:

SN	Operation	Description
1	Insertion at beginning	Adding a node into a circular singly linked list at the beginning.
2	Insertion at the end	Adding a node into a circular singly linked list at the end.
3	Deletion at beginning	Removing the node from the circular singly linked list at the beginning.
4	Deletion at the end	Removing the node from the circular singly linked list at the end.
5	Searching	Compare each element of the node with the given item and return the location at which the item is present in the list; otherwise return null.
6	Traversing	Visiting each element of the list at least once in order to perform some specific operation.

Insert at the beginning of a Circular Linked List

There are two scenarios in which a node can be inserted in a circular singly linked list at the beginning. Either the node will be inserted in an empty list or the node is to be inserted in an already filled list.

Firstly, allocate the memory space for the new node by using the malloc method of C language.

A) **In the first scenario,** the condition **head == NULL** will be true. Since, the list in which we are inserting the node is a circular singly linked list, therefore the only node of the list (which is just inserted into the list) will point to itself only. We also need to make the head pointer point to this node. This will be done by using the following statements.

```
if(head == NULL)
        {
            head = newNode;
            newNode-> next = head;
        }
```

B) In the second scenario, the condition head == NULL will become false which means that the list contains at least one node. In this case, we need to traverse the list in order to reach the last node of the list. This will be done by using the following statement.

```
temp = head;
while(temp->next != head)
   temp = temp->next;
```

With reference to the second scenario; at the end of the loop, the pointer temp would point to the last node of the list. Since, in a circular singly linked list, the last node of the list contains a pointer to the first node of the list. Therefore, we need to make the next pointer of the last node point to the head node of the list and the new node which is being inserted into the list will be the new head node of the list therefore the next pointer of **temp** will point to the new node **newNode**.

This will be done by using the following statements.

temp -> next = newNode;

The next pointer of temp will point to the existing head node of the list.

newNode->next = head;

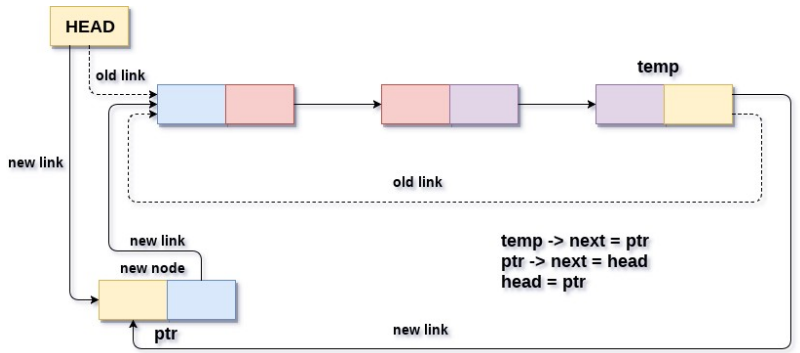
Now, make the new node **newNode**, the new head node of the circular singly linked list.

head = newNode;

Algorithm

- **Step1: CREATE** SET NEW_NODE
- **Step 2:** SET NEW_NODE -> DATA = VAL
- **Step 3:** SET TEMP = HEAD
- **Step 4:** Repeat Step 5 while TEMP -> NEXT ! = HEAD
- **Step 5:** SET TEMP = TEMP -> NEXT

  [END OF LOOP]
- **Step 6:** SET NEW_NODE -> NEXT = HEAD
- **Step 7:** SET TEMP → NEXT = NEW_NODE
- **Step 8:** SET HEAD = NEW_NODE
- **Step 9:** EXIT



Insertion into circular singly linked list at beginning

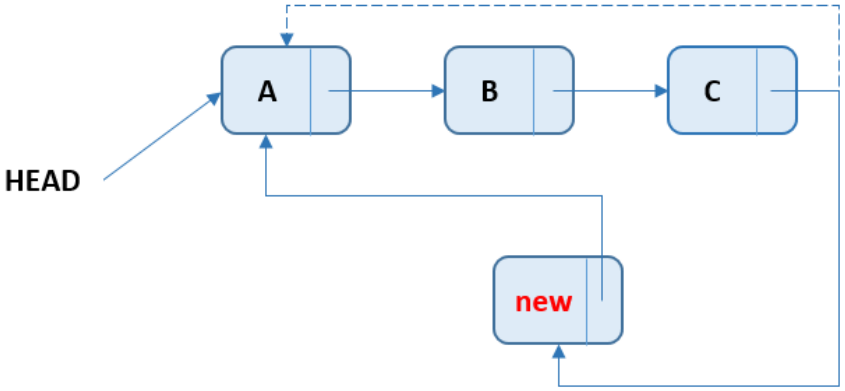
```
#include<iostream>
using namespace std;
struct Node {
  int data;
  Node *next;
 };
 Node *head=NULL;
```

```
        void addAtStart( int data){
            //Create new node
           Node *newNode = new Node();
           newNode->data=data;
           //Checks if the list is empty.
           if(head == NULL) {
                //If list is empty, both head and tail would point to new node.
               head = newNode;
               newNode->next = head;
           }
           else {
             //Store the head into temporary node
            Node *temp = head;
            while(temp->next != head)
             temp = temp->next;
           //The next pointer of temp will point to the new node newNode.
             temp -> next = newNode;
             //The newNode  will point to the existing head of the circular singly linked list.
             newNode->next = head;
             //Now, make the new node newNode, the new head node of the circular singly linked
list.
             head = newNode;

           }
       }
```

Inserting a node at the end of a circular linked list

In this method, a new node is inserted at the end of the circular singly linked list. For example - if the given List is 10->20->30 and a new element 100 is added at the end, the List becomes 10->20->30->100.

Inserting a new node at the end of the circular singly linked list is very easy. First, a new node with the given element is created. It is then added at the end of the list by linking the last node and head node to the new node.



Insert at the end of the Linked List

```
//Add new element at the end of the list
 void push_back(int newElement) {
   Node *newNode = new Node();
   newNode -> data = newElement;
   newNode -> next = NULL;
   if (head == NULL) {
     head = newNode;
     newNode -> next = head;
   } else {
     Node * temp = head;
     while (temp -> next != head)
       temp = temp -> next;
     temp -> next = newNode;
     newNode -> next = head;
   }
 }
```

Delete at the beginning

Here is the function to perform deletion at the beginning as it is self explanatory since we have talked about in the previous types of a linked list.

```
// Function to delete last node of
// Circular Linked List
void DeleteLast()
{
        Node *current = head, *temp = head,
*previous;
        // check if list doesn't have any node
        // if not then return
        if (head == NULL) {
                printf("\nList is empty\n");
                return;
        }
        // check if list have single node
        // if yes then delete it and return
        if (current->next == current) {
                head = NULL;
                return;
        }
        // move first node to last
        // previous
        while (current->next != head) {
                previous = current;
                current = current->next;
        }
        previous->next = current->next;
        head = previous->next;
        free(current);
        return;
}
```

Delete at the end

Here is the function to perform deletion at the end as it is self explanatory since we have talked about in the previous types of a linked list.

```
// Function delete First node of Circular Linked List
void DeleteFirst()
{
        Node *previous = head, *next = head;
        // check list have any node
        // if not then return
        if (head == NULL) {
        cout<<"List is empty\n";
                return;
        }
        // check list have single node
        // if yes then delete it and return
        if (previous->next == previous) {
                head = NULL;
                return;
        }
        // traverse second to first
        while (previous->next !=head) {
                previous = previous->next;
                next = previous->next;
        }
// now previous is last node and  next is first node of list
        // first node(next) link address
        // put in last node(previous) link
        previous->next = next->next;
        // make second node as head node
        head = previous->next;
        free(next);
        return;
}
```

Traversing the Circular Linked List

```
void PrintList() {
  Node * temp = head;
  int sum=0;
  if (temp != NULL) {
    cout << "The list contains: ";
    while (true) {
    sum+=temp->data;
      cout << temp -> data << " ";
      temp = temp -> next;
      if (temp == head)
        break;
    }
    cout << endl;
    cout<<"The total saved is "<<sum<<endl;
  } else {
    cout << "The list is empty.\n";
  }
}
```

Counting the element of a Circular Linked List

```
// Function return number of nodes present in list
int Length()
{
        struct Node* current = head;
        int count = 0;
        // if list is empty simply return length zero
        if (head == NULL) {
        return 0;
        }

        // traverse first to last node
        else {
        do {
        current = current->next;
        count++;
        } while (current != head);
        }
        return count;
}
```

Main method

```
int main() {
  //Add three elements at the end of the list.
  push_back(1000);
  PrintList();
  push_back(2000);
  push_back(3000);
  push_back(50000);
  PrintList();
  addAtStart(8);
  addAtStart(7);
  addAtStart(6);
   PrintList();
   DeleteFirst();
   DeleteLast();
   PrintList();
  return 0;
}
```

Exercises

1. Using Floyd's Tortoise and Hare algorithm, determine if a linked list has a loop.
2. Write a console C++ program to implement the following data structures' operation: linked list of Student(code, name and school). The program should continue to run and exit only if the user enters 0.

Note the following operations to be implemented are displayed as Menu as follow:

- 1.add at the beginning
- 2.Add at the end
- 3.Display student
- 4.Update a student name when his/her code is provided
- 5.delete the last student.
- 6.Delete at the beginning